WEEK-4 : CUSTOMER AND PRODUCT ORDER ANALYSIS.

Scenario

Art Work Shop manages customer orders and product sales using a database system. The database stores customer, category, product, order and order details information. The reports help to view customer order history, product-wise order quantities, total customer spending and average order value. These reports make it easier to analyse the workshop's orders and customer purchases.

Create Table – Orders

The Orders table is created to store the details of customer orders. It contains the order ID, customer ID, order date, total amount and order status. Primary key, foreign key, NOT NULL and CHECK constraints are included while creating the table.

```
SQL> CREATE TABLE Orders (
  2      Order_ID NUMBER PRIMARY KEY,
  3      Customer_ID NUMBER NOT NULL,
  4      Order_Date DATE NOT NULL,
  5      Total_Amount NUMBER(10,2) NOT NULL,
  6      Order_Status CHAR(20) NOT NULL,
  7      CONSTRAINT fk_orders_customer
  8          FOREIGN KEY (Customer_ID)
  9          REFERENCES Customer(Customer_ID),
 10      CONSTRAINT chk_order_status
 11          CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered'))
 12  );

Table created.
```

Create Table – Order_Details

The Order_Details table is created to store the details of products included in each order. It contains the order detail ID, order ID, product ID, quantity and price. Primary key, foreign key, NOT NULL and CHECK constraints are included while creating the table.

```
SQL> CREATE TABLE Order_Details (
  2      Order_Detail_ID NUMBER PRIMARY KEY,
  3      Order_ID NUMBER NOT NULL,
  4      Product_ID VARCHAR2(10) NOT NULL,
  5      Quantity NUMBER NOT NULL,
  6      Price NUMBER(10,2) NOT NULL,
  7      CONSTRAINT fk_od_order
  8          FOREIGN KEY (Order_ID)
  9          REFERENCES Orders(Order_ID),
 10      CONSTRAINT fk_od_product
 11          FOREIGN KEY (Product_ID)
 12          REFERENCES Product(Product_ID),
 13      CONSTRAINT chk_od_quantity
 14          CHECK (Quantity > 0),
 15      CONSTRAINT chk_od_price
 16          CHECK (Price > 0)
 17  );

Table created.
```

Establish Relationship Using Primary Key and Foreign Key

The relationship between the tables is established using Primary Key (PK) and Foreign Key (FK).

- **CUSTOMER_ID in Customer → CUSTOMER_ID in Orders**
- **ORDER_ID in Orders → ORDER_ID in Order_Details**
- **PRODUCT_ID in Product → PRODUCT_ID in Order_Details**

These relationships connect customer, order and product information in the database and maintain referential integrity.

Apply Constraints

The database tables are created with different constraints to maintain data accuracy and integrity.

- **Primary Key (PK):** Used to uniquely identify each record in a table.
- **Foreign Key (FK):** Used to establish relationships between tables.
- **NOT NULL:** Ensures that required fields cannot be left empty.
- **CHECK Constraint:** Ensures that only valid values are entered into the tables.

These constraints help maintain the accuracy, consistency and reliability of the database.

Inserting Values

```
SQL> INSERT ALL
  2  INTO Orders VALUES (1001, 2, TO_DATE('2026-10-01','YYYY-MM-DD'), 2500, 'Pending')
  3  INTO Orders VALUES (1002, 3, TO_DATE('2026-10-02','YYYY-MM-DD'), 4500, 'Shipped')
  4  INTO Orders VALUES (1003, 4, TO_DATE('2026-10-03','YYYY-MM-DD'), 3400, 'Delivered')
  5  INTO Orders VALUES (1004, 5, TO_DATE('2026-10-04','YYYY-MM-DD'), 7500, 'Pending')
  6  INTO Orders VALUES (1005, 6, TO_DATE('2026-10-05','YYYY-MM-DD'), 1500, 'Shipped')
  7  SELECT * FROM DUAL;

5 rows created.
```

```
SQL> INSERT ALL
  2  INTO Order_Details VALUES (1, 1001, 'P001', 1, 2500)
  3  INTO Order_Details VALUES (2, 1002, 'P002', 2, 2250)
  4  INTO Order_Details VALUES (3, 1003, 'P003', 1, 3400)
  5  INTO Order_Details VALUES (4, 1004, 'P004', 3, 2500)
  6  INTO Order_Details VALUES (5, 1005, 'P005', 1, 1500)
  7  SELECT * FROM DUAL;

5 rows created.
```

```
SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
      1001           2 01-OCT-26         2500 Pending
      1002           3 02-OCT-26         4500 Shipped
      1003           4 03-OCT-26         3400 Delivered
      1004           5 04-OCT-26         7500 Pending
      1005           6 05-OCT-26         1500 Shipped
```

```
SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID   ORDER_ID PRODUCT_ID   QUANTITY      PRICE
--------------- ---------- ---------- ---------- ----------
              1       1001 P001                1       2500
              2       1002 P002                2       2250
              3       1003 P003                1       3400
              4       1004 P004                3       2500
              5       1005 P005                1       1500
```

1. Update Record

The existing order record is updated to modify the total amount.

```
SQL> UPDATE Orders SET Order_Status = 'Shipped' WHERE Order_ID = 1001;

1 row updated.
```

2. Delete Record

An existing order record is deleted from the Orders table. The corresponding record in `Order_Details` must be deleted first because of the foreign key relationship.

```
SQL> DELETE FROM Order_Details WHERE Order_Detail_ID = 1;

1 row deleted.

SQL> DELETE FROM Orders WHERE Order_ID = 1001;

1 row deleted.
```

Report 1 – Customer Order History

This report displays the order history of each customer, including their name, order ID, order date, total amount and order status.

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE,
  2         O.TOTAL_AMOUNT, O.ORDER_STATUS
  3  FROM Customer C
  4  JOIN Orders O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID;

CUSTOMER_NAME     ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
--------------- ---------- --------- ------------ ---------------
Priya                 1002 02-OCT-26         4500 Shipped
Karthik               1003 03-OCT-26         3400 Delivered
Divya                 1004 04-OCT-26         7500 Pending
Rahul                 1005 05-OCT-26         1500 Shipped
```

Report 2 – Product-wise Order Report

This report displays the total quantity ordered for each product. It helps analyse products based on their order quantities.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, O.ORDER_ID,
  2         O.ORDER_DATE, OD.QUANTITY, OD.PRICE,
  3         (OD.QUANTITY * OD.PRICE) AS TOTAL_PRICE
  4  FROM Product P
  5  JOIN Order_Details OD ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  JOIN Orders O ON OD.ORDER_ID = O.ORDER_ID;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
--
  ORDER_ID ORDER_DAT   QUANTITY      PRICE TOTAL_PRICE
---------- --------- ---------- ---------- -----------
P002
Mobile
      1002 02-OCT-26          2       2250        4500

P003
Watch
      1003 03-OCT-26          1       3400        3400

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
--
  ORDER_ID ORDER_DAT   QUANTITY      PRICE TOTAL_PRICE
---------- --------- ---------- ---------- -----------

P004
```

```
--
  ORDER_ID ORDER_DAT   QUANTITY      PRICE TOTAL_PRICE
---------- --------- ---------- ---------- -----------

P004
Headphones
      1004 04-OCT-26          3       2500        7500

P005
Bag

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
--
  ORDER_ID ORDER_DAT   QUANTITY      PRICE TOTAL_PRICE
---------- --------- ---------- ---------- -----------
      1005 05-OCT-26          1       1500        1500
```

Report 3 – Customer Purchase Analysis

This report analyses customer purchases by identifying customers with the maximum number of orders, calculating the total amount spent by each customer, and finding their average order value.

Customers with Maximum Orders

Identifies the customers who have placed the maximum number of orders.

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  2  FROM Customer C
  3  JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID
  4  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  5  HAVING COUNT(O.ORDER_ID) = (
  6      SELECT MAX(ORDER_COUNT)
  7      FROM (
  8          SELECT COUNT(ORDER_ID) AS ORDER_COUNT
  9          FROM Orders
 10          GROUP BY CUSTOMER_ID
 11      )
 12  );

CUSTOMER_NAME   TOTAL_ORDERS
--------------- ------------
Priya                      1
Rahul                      1
Karthik                    1
Divya                      1
```

Total Spending by Each Customer

Calculates the total amount spent by each customer.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2  SUM(Orders.TOTAL_AMOUNT) AS TOTAL_SPENDING
  3  FROM Customer
  4  JOIN Orders
  5  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  6  GROUP BY Customer.CUSTOMER_ID, Customer.CUSTOMER_NAME;

CUSTOMER_NAME   TOTAL_SPENDING
--------------- --------------
Priya                     4500
Rahul                     1500
Karthik                   3400
Divya                     7500
```

Average Order Value

Calculates the average order value for each customer.

```
SQL> SELECT Customer.CUSTOMER_NAME,
  2  AVG(Orders.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  3  FROM Customer
  4  JOIN Orders
  5  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID
  6  GROUP BY Customer.CUSTOMER_ID, Customer.CUSTOMER_NAME;

CUSTOMER_NAME   AVERAGE_ORDER_VALUE
--------------- -------------------
Priya                          4500
Rahul                          1500
Karthik                        3400
Divya                          7500
```